PROJECT STUDY GUIDE · 01

AiMenu 智能点餐系统 Agent 学习指南

从请求入口到模型决策、工具调用、数据库写入与流式返回
结合真实项目代码逐段理解 LangChain、LangGraph、Prompt、FastAPI 与 MCP

阅读目标

读完后能跟踪一条用户消息如何进入 Agent；理解模型为什么调用工具、参数 Schema 如何形成、工具如何访问 MySQL/Milvus、FastAPI 如何将消息以 SSE 发送到 Vue 页面；并能识别当前实现的边界和安全注意事项。

基于本仓库当前源码整理 · 面向项目学习者
说明：示例值与能力边界以源码为准，不代表尚未实现的功能。

阅读导航与核心结论

这不是一份泛讲 Agent 的介绍，而是按当前仓库的真实调用顺序解释。建议先读“端到端链路”，再按 `api/main.py → agent/langchain_assistant.py → agent/prompts/system_prompt.txt → ui/src/app.vue` 的顺序对照代码。

1. 项目里的 Agent 是什么，实际消息路径如何走。
2. Chat model、Prompt、ReAct Agent、图执行和 checkpoint。
3. 三个业务工具：特色菜、口味检索与 MySQL 回退、预约写入。
4. FastAPI、异步流、SSE 事件协议及 Vue 消费逻辑。
5. MCP 的职责、可选配置、连接失败处理和当前边界。
6. 可靠性与安全审阅：哪些规则只是 Prompt、哪些行为由代码保证。
7. 带着问题逐步完成代码阅读和实验的学习路线。

> **一句话概括：**FastAPI 接收用户文本，LangGraph 的 ReAct Agent 让配置的聊天模型决定回答还是调用工具；工具访问 MySQL 或可选的向量服务/MCP；Agent 把最终文本流交给后端，后端再以 SSE 传给 Vue。

```
Vue 页面：sendChatQuery()
  → POST /api/chat（浏览器 Fetch + ReadableStream）
  → Vite 代理移除 /api 前缀
  → FastAPI：POST /chat（ChatRequest、StreamingResponse）
  → assistant_query(query)
  → create_agent()（只在首次调用时构造）
  → LangGraph create_react_agent(model, tools, prompt, checkpointer)
  → 模型回答，或产生 tool call → Python tool → MySQL / Milvus / MCP
  → Agent 组织最终回答
  → LangGraph message chunks → 后端过滤 ToolMessage → SSE data JSON
  → Vue 按行解析 token/error 并更新聊天气泡
```

接口代理目标依当前 `ui/vite.config.js` 配置；本指南不假设代理一定使用某个固定浏览器端口。

1. Agent 与普通 LLM 调用的区别

普通模型调用

普通调用通常是一问一答：应用把系统指令和用户问题发给模型，拿回一段文字。即使模型说“我查询了数据库”，如果应用没有实现相应代码，这句话也不会真的执行数据库查询。

本项目的 Agent

本项目在模型外面增加一个由 LangGraph 管理的循环：模型可以发出结构化工具调用；运行时根据工具名与参数，执行 Python 函数；工具结果返回给模型；模型再决定继续调用工具还是给最终答复。这个“模型决策 + 程序执行 + 结果回传”的闭环才是 Agent 行为。

> 用户：有哪些不辣的素菜?
> 模型：决定调用 `user_flavor_search(user_query="不辣的素菜")`
> 运行时：执行工具，读取 Milvus 或 MySQL 在售菜单
> 工具结果：菜单字段/检索文本返回给 Agent
> 模型：根据 spice_level、is_vegetarian 筛选并组织回复
> 后端：只把 assistant 文本 token 流式送给网页

需要区分“模型决定调用工具”与“Python 执行工具”：模型本身不会直接连接 MySQL，也不会访问本地文件。工具的副作用、权限和事务都由 Python 实现决定。

本项目使用的 Agent 框架

`create_agent()` 中调用 `langgraph.prebuilt.create_react_agent`。LangChain 提供聊天模型、工具封装和通用接口；LangGraph 负责以图/状态机形式推进模型节点和工具节点、传递消息状态，并通过 checkpointer 管理线程状态。项目没有自行手写完整 ReAct while 循环，预构建 Agent 承担了这个控制流。

核心实现：`agent/langchain_assistant.py → create_agent()`，文件末尾。

2. 模型配置、Prompt 与 Agent 初始化

2.1 聊天模型客户端

`create_chat_model()` 从环境变量读取 `OPENAI_API_KEY`、`OPENAI_BASE_URL`、`OPENAI_MODEL`，用 LangChain 的 `ChatOpenAI` 构造聊天模型。这里的类名表示 OpenAI 协议兼容客户端，不等于模型必须来自 OpenAI；实际供应方、模型名称和 endpoint 由配置决定。

配置	用途	注意点
`OPENAI_API_KEY`	上游模型服务鉴权密钥	只放在根目录本机 .env 或密钥管理中，不要写入源码、日志和 PDF。
`OPENAI_BASE_URL`	OpenAI 兼容 API 的基地址	须包含服务商要求的路径，且与密钥的地域/账号匹配。
`OPENAI_MODEL`	上游识别的模型 ID	须是该账号和 endpoint 支持的模型。源码当前默认值是 claude-sonnet-5；设置环境变量后以环境变量为准。
temperature=0.7	控制生成采样随机性	温度不构成事实校验，事实仍应以工具结果为准。
timeout=30、max_retries=1	请求超时与 SDK 重试限制	重试仍可能增加一次调用耗时；SSE 把异常转成安全错误事件。

客户端显式以 certifi CA 文件建立 SSL context，仍执行证书与主机名校验。同步和异步客户端在 FastAPI lifespan 结束时关闭；它们是为了复用连接池，而不是每条消息重新创建 TLS 连接。

2.2 Prompt 作为稳定的系统指令

`create_agent()` 用 UTF-8 读取 `agent/prompts/system_prompt.txt`，然后把文本作为 `prompt=system_prompt` 交给 Agent。Prompt 定义模型的角色、餐厅业务范围、要收集的预订信息、工具何时使用、不可捏造等约束。

当前 Prompt 还指定菜单/口味问题必须调用 `user_flavor_search`，特色菜查询调用 `search_main_dishes`；要求根据真实工具结果回答，并提醒“点菜/推荐”不是预订，不可声称已经下单。

> **Prompt 不等于访问控制：** “预订要二次确认”是给模型看的指令；`make_reservation` 函数并没有独立的确认令牌或确认状态校验。要用于真实业务，应由服务端状态机或 API 强制确认，而不能只信模型遵守 Prompt。

2.3 create_react_agent 参数

```
agent = create_react_agent(
    model=llm,
    tools=tools,
    prompt=system_prompt,
    checkpointer=MemorySaver(),
)
```

- `model`：决定如何回答、是否调用工具、生成工具参数。
- `tools`：允许 Agent 请求的能力清单。
- `prompt`：角色、业务约束和工具使用策略。
- `checkpointer`：在 thread_id 对应的线程中保存 LangGraph 状态。

源码位置：`agent/langchain_assistant.py → create_chat_model()` 与 `create_agent()`。

3. LangChain Tools：模型可调用的业务能力

本项目通过 `from langchain.tools import tool` 和 `@tool` 把 Python 函数包装成 LangChain 工具。函数名、文档字符串和参数类型/Schema 会成为模型理解“何时调用、如何填写参数”的工具描述；实际函数体仍由 Python 运行。

工具	输入	做什么	数据源
`search_main_dishes`	无业务参数	查当前可供应特色菜；如果旧表没有 is_featured 列，则兼容地查可供应菜品。	MySQL menu_items
`user_flavor_search`	`user_query: str`	有可用本地模型及真实 Milvus URL 时向量检索；未配置、错误或无结果时回退到在售 MySQL 菜单。	可选 BGE/Milvus，回退 MySQL
`make_reservation`	6 个预订字段	参数化 INSERT 预约记录并提交事务。	MySQL reservation_order

3.1 参数 Schema 与校验

`ReservationToolArgsInfo` 是 Pydantic `BaseModel`，借助 `@tool(args_schema=...)` 明确规定人数、儿童人数、到达时间、座位偏好、主菜偏好和备注的描述。这些类型/描述帮助模型构造结构化参数，框架也会在调用函数前解析输入。

Schema 解析不等于完整业务校验：当前字段没有设置人数范围、儿童数不得大于总人数、时间必须为未来等验证器；字符串 `arrival_time` 最终交给数据库，额外约束尚未实现。

3.2 工具结果回到模型

LangGraph 运行工具后生成工具结果消息（LangChain 的 `ToolMessage`）。该结果供 Agent 下一步推理使用。后端在流式转发层显式跳过 `ToolMessage`，所以网页不会把内部数据库结果原样当作 assistant 发言显示；Agent 会先看到工具结果，再生成面向用户的回答。

3.3 安全设计要点

- 查询使用 SQLAlchemy `text()` 和绑定参数，预订写入采用命名参数，而不是把用户值拼接进 SQL。
- `user_flavor_search` 返回真实数据库字段，价格以字符串保留精度；服务失败时返回通用错误并在服务端记录详细异常。
- `make_reservation` 执行 INSERT 后显式 `commit()`；失败时返回失败原因。目前函数返回错误文本时仍可能含底层异常细节，生产部署需审查日志/错误脱敏和权限。
- 当前没有“取消预约”工具，也没有“提交菜品订单”的工具。购物车 UI 状态不等于服务端订单落库。

4. 工具一：菜单与特色菜的真实数据路径

4.1 连接池如何延迟创建

`mysql_connection()` 使用 SQLAlchemy 的 `URL.create()` 组合 MySQL 主机、用户名、密码、端口和数据库名；首次调用时才创建全局 SQLAlchemy Engine，之后复用连接池。密码由 URL 对象处理转义，`pool_pre_ping=True` 会在借出连接时探测连接是否失效。

4.2 特色菜工具

`search_main_dishes()` 检查表字段是否存在 `is_featured`，按字段决定 WHERE 条件，再过滤 `is_available=1`。这是对旧版数据库结构的兼容逻辑。结果按字段名映射为中文标签，方便模型阅读。该函数给模型的是菜品事实；最终怎么摘要仍由模型生成。

4.3 普通菜单/口味工具

`load_available_menu()` 查询所有可售菜单字段，其中 `spice_level=0` 表示不辣，`is_vegetarian` 表示素食。返回结构清楚标注 `source="mysql"`，并告诉模型不能捏造菜名或价格。

`search_main_dishes` 与 `user_flavor_search` 是供 Agent 工具调用的函数；页面右侧菜单则走独立的 FastAPI `/menu/list` 路由。二者都查 MySQL，但不应把 Agent 工具与前端 API 混为一谈。

4.4 可选向量检索与回退

```
用户的自然语言偏好
  → get_embeddings(): 加载 models/bge-m3
  → embed_query(): 偏好转换成数值向量
  → get_milvus_client().search(collection="menu_items")
  → 有结果: 返回 source="milvus" 的文本
  → 未配置模型 / 占位 URI / 连接失败 / 无结果
  → load_available_menu(): 读 MySQL 当前可售菜单
  → Agent 依据辣度、素食、食材等字段筛选
```

这样基础口味推荐不以 Milvus 为前提。Milvus 同步脚本 `agent/milvus_data_sync.py` 是另一条离线数据准备链路；它目前包含删除并重建 collection 的行为，运行前必须审查目标服务和数据备份。不要把该同步脚本当作安全的 Agent 工具或普通无副作用导入。

5. 工具二：预订写入与事务

5.1 从 Schema 到 INSERT

`make_reservation` 接收与 Schema 对应的参数，将 Python 参数映射为 SQLAlchemy 命名绑定参数，并执行：

```
INSERT INTO reservation_order
(num_people, num_children, arrival_time,
 seat_preference, main_dish_preference, other_comments)
VALUES
(:num_people, :num_children, :arrival_time,
 :seat_preference, :main_dish_preference, :other_comments)
```

绑定参数将 SQL 结构与用户数据分离；数据库 INSERT 成功后调用 `conn.commit()`，让记录正式保存。函数捕获错误后返回“预订失败”结果，避免在正常工具流程中直接崩溃。

5.2 预订流程应该如何理解

1. 模型从对话识别用户明确需要预订。
2. Prompt 要求收集人数、时间、座位、主菜和备注。
3. 缺信息时模型应继续询问。
4. Prompt 要求收齐后先二次确认。
5. 确认后模型才应该调用 `make_reservation` 写入 MySQL。
6. 服务端查询 `/reservation/list` 读预约列表。

> **重要实现差异：**步骤 1–4 的决策主要由模型 + Prompt 承担。当前 Python 函数收到有效工具参数就能写入；数据库没有确认字段，后端没有确认 token，也没有独立预约 API 的人工确认门禁。练习前要理解“指令要求”与“不可绕过的业务规则”差异。

5.3 生产级预约还需要什么

身份认证与预约所有权校验、服务端确认状态、输入约束、重复提交幂等键、预约时间/容量检查、并发处理、用户取消状态（非硬删除）、审计记录、隐私与日志脱敏。当前列表路由返回预订记录，没有身份隔离机制，因此只适合本地学习演示，不应直接暴露到公网。

6. LangGraph 状态、消息与记忆

6.1 Agent 的 state 是什么

预构建 ReAct Agent 的核心状态包含消息序列：user 消息、assistant 决策/工具调用消息、ToolMessage 结果，以及 assistant 最终答复。图每运行一轮，状态就沿模型节点与工具节点推进。 `astream(..., stream_mode="messages")` 把新产生的消息/消息块逐步发出，而不是等待完整回答。

6.2 Checkpointer 和 thread_id

`MemorySaver()` 是内存型 checkpoint，按配置的线程 ID 保存图状态。 `assistant_query()` 当前每次请求使用同一个常量 `thread_id=123` ，并输入一条新用户消息。LangGraph 有机会将同一线程的历史消息累积进状态；这有利于多轮上下文，但也带来明显限制。

- 所有用户共用一个 thread_id，不能区分访客；多用户请求可能共享上下文，造成隐私串话。
- 内存 checkpoint 在进程重启后清空，不是持久化会话存储。
- 当前前端没有显式向后端传稳定的用户/会话 ID。
- 生产设计应使用每个会话隔离、授权且不可由其他用户猜测的 thread_id，并对历史设定保留与清理策略。

6.3 静态图中的动态数据

`create_agent()` 将 Agent 缓存在模块全局变量 `agent` 。Agent 初始化时固定模型、Prompt、工具列表和 checkpointer；每次聊天把当前日期/星期作为一条额外 system 消息传入 state。因此改了 Prompt 或环境配置后通常要重启后端，才能保证已缓存的 Agent 使用新设置。

源码位置：`agent/langchain_assistant.py` → `assistant_query()`、`create_agent()`；FastAPI 生命周期在 `api/main.py`。

7. FastAPI 如何把 Agent 接到网页

7.1 FastAPI 路由

`api/main.py` 建立 `app = FastAPI(lifespan=lifespan)`。`POST /chat` 接收 Pydantic `ChatRequest`（JSON 的 `{"query":"..."}`），拒绝空字符串；非空输入将异步生成器 `assistant_query(query)` 包装在 Starlette 的 `StreamingResponse` 中。

FastAPI 负责 HTTP 层：路径匹配、请求体解析、响应头与异步流。它不是 Agent，也不会自行做模型推理。

7.2 异步生成器如何逐片产出

`assistant_query()` 是 `async def` 生成器：内部用 `async for` 消费 Agent 的异步 `astream`，每拿到一个非空 assistant 文本块就 `yield` 一条 SSE 帧。工具消息不转发；多模态内容列表中仅提取 text block。出错时记录服务端异常日志，并返回结构化的安全错误信息。

7.3 SSE 线上格式

```
data: {"content":"您好","type":"token"}\n\n
data: {"content":", 欢迎光临","type":"token"}\n\n
```

`text/event-stream` 是服务端事件流 MIME 类型。每个事件以 `data:` 开始，空行分隔。前端用 Fetch 获取 Response.body 的 Reader，TextDecoder 解码字节，缓存不完整的半行、以换行符拆事件，再 JSON.parse。

7.4 Vue 如何更新气泡

`ui/src/api/index.js` 的 `sendStreamMessage()` 负责 fetch 和 SSE 解码；`ui/src/app.vue` 的 `sendChatQuery()` 先添加用户消息、显示等待状态，再收到 `type="token"` 时把文本追加进 assistant 消息。错误事件显示可读提示，收到结束回调后清除加载状态并更新连接状态标签。

这是一条流式 UI 链路，而非 WebSocket：浏览器发起普通 HTTP POST，服务器只在该请求期间持续写响应内容。前端异常要区分网络层 HTTP 错误与 SSE 内嵌的 `type="error"`。

7.5 lifespan 资源清理

FastAPI lifespan 在服务退出时调用 `close_model_clients()`，关闭共享的同步/异步模型 HTTP 客户端并清空 Agent 缓存。这能避免长期运行进程退出时连接池不释放；数据库引擎目前并没有在这里 dispose。

8. MCP：把外部工具接入 Agent

8.1 MCP 的角色

MCP（Model Context Protocol）定义了客户端与 MCP server 之间发现和调用工具的协议。MCP server 实现地图等外部能力；本项目通过 LangChain MCP adapter 充当 MCP client，取回工具定义，再把它们与本地 Python Tools 一起交给 ReAct Agent。模型仍负责判断何时调用，工具执行由 MCP server 负责。

8.2 当前连接代码

```
mcp_url = os.getenv("AMAP_MCP_URL")
if mcp_url:
    client = MultiServerMCPClient(
        connections={"amap_map": {"transport": "sse", "url": mcp_url}}
    )
    tools += await asyncio.wait_for(client.get_tools(), timeout=5)
```

`AMAP_MCP_URL` 存在时，`MultiServerMCPClient` 以 SSE transport 配置名为 `amap_map` 的 server，并通过 `get_tools()` 发现工具。发现到的 MCP 工具追加进本地工具清单。用 `wait_for(..., timeout=5)` 防止 Agent 初始化无限等待；失败时写 warning，并继续使用本地 MySQL 工具。

8.3 当前 MCP 的边界

- 未设置 `AMAP_MCP_URL` 时不会连接地图 MCP，也不会影响基础 Agent 初始化。
- 设置 URL 不等于地图服务一定可用；还需要网络、正确 SSE 地址及 server 提供兼容工具。
- 项目未在对话中询问或请求用户授权地图工具操作的独立确认框；第三方 MCP 服务的工具语义必须先审查。
- MCP 与普通本地 Tools 共用 Agent 的模型决策；应按工具权限对 MCP server 做最小授权并限制超时。
- FAQ Redis、MySQL 菜单 API、Milvus 检索与 MCP 是不同数据/工具通路。不是每条聊天请求都经过它们。

9. Prompt、工具和 Agent 的设计原则

9.1 Prompt 要告诉模型什么

- **身份/范围：**餐厅助手，只回答有资料支撑的餐厅问题。
- **工具选择：**特色菜用 `search_main_dishes` ，菜单偏好用 `user_flavor_search` 。
- **缺信息处理：**预订信息不完整时继续询问。
- **确认/边界：**预订写入前请求用户确认；推荐不能冒充已下单。
- **事实来源：**价格、食材、辣度以工具结果为准；没有结果时明确说明。

9.2 Tool 说明为什么重要

模型依赖工具名称、docstring、Schema 描述来挑选工具和生成参数。函数名太泛、参数描述含混，会造成误调用或不合适参数。应让工具描述讲清适用场景、返回格式、单位、失败含义；在 Python 侧做硬性验证，而非寄希望于 Prompt。

9.3 何时不该调用工具

固定营业时间已写在系统 Prompt 中，回答此问题无需联网/数据库工具；菜品价格则应查真实菜单；需要外部地图信息时才需要 MCP（且服务已配置）。尽量避免为简单闲聊调用数据库、同时保护写操作工具。

9.4 写工具的三层防护

1. 工具描述帮助模型正确选择；
2. Schema/代码验证参数并检查授权/状态；
3. 数据库事务、幂等、权限与审计保护最终副作用。

Schema 是结构校验，Prompt 是模型行为指引，服务端业务校验才是可信的执行边界。三个层次缺一不可。

10. 建议的源码学习路线

阶段 A：先理解一次聊天

1. 打开 `ui/src/app.vue`，阅读输入框发送按钮、`sendChatQuery()`。
2. 打开 `ui/src/api/index.js`，阅读 `sendStreamMessage()` 的 Reader、Decoder、buffer。
3. 打开 `api/main.py`，从 `@app.post("/chat")` 跟到 `StreamingResponse`。
4. 打开 `agent/langchain_assistant.py`，看 `assistant_query()` 如何消费 Agent 流。

阶段 B：理解 Agent 的决策与能力

1. 先读 `agent/prompts/system_prompt.txt`，把每条要求对应到代码或工具。
2. 检查 `@tool`、函数 docstring 和 Pydantic args_schema。
3. 对照 `create_agent()` 的 model、tools、prompt、checkpointer。
4. 追踪一个特色菜 tool call：Assistant 的工具调用 → Python SQL → ToolMessage → Assistant 最终回答。

阶段 C：理解外部服务与生命周期

1. 读 `mysql_connection()`，理解 Engine/连接池与 SQL。
2. 读 `get_embeddings()`、`get_milvus_client()` 和 `user_flavor_search()` 的回退分支。
3. 读 MCP 环境变量和 `get_tools()` 的可选加载逻辑。
4. 读 `api/main.py` 的 lifespan 关闭模型 HTTP 客户端。

阶段 D：安全地观察调用

使用本地开发环境，观察后端日志与浏览器网络请求：先问营业时间（无工具），再问特色菜（MySQL 工具），再问不辣推荐（Milvus 或 MySQL 回退）。写预约实验需只在个人测试库进行，确认不会创建真实业务记录；不要直接执行会清空 collection 的数据同步脚本。

11. 项目能力清单与常见误区

能力/依赖	当前源码中的状态	学习时要记住
Agent 多轮工具调用	已实现，LangGraph ReAct	LangGraph 驱动循环，不是仅由 prompt 生成答案。
模型供应方/模型名	由 .env 的 OPENAI_* 环境变量配置	ChatOpenAI 是协议适配，不代表供应方；要核实实际模型 ID。
特色菜查询	MySQL 查询在售菜单及特色标记	页面菜单接口与 Agent tool 是两条独立路由。
口味向量检索	可选 BGE-M3 + Milvus，并回退 MySQL	没有模型/服务时仍可由 MySQL 提供菜单，但语义检索能力不同。
预订写入	Agent tool 插入 MySQL	二次确认主要依靠 Prompt，没有服务端确认 token 门禁。
取消预约	未实现	无取消工具、路由或状态字段。
真实点餐订单/支付	未实现	前端购物车状态不是数据库订单，也没有订单 API。
MCP 地图	可选；由 AMAP_MCP_URL 加载	URL 未配置/连接失败不会阻断本地菜单工具。
FAQ 推荐	FastAPI 独立查询 Redis + 字符串相似度	不是 Agent 工具调用，不经过 LangGraph 模型循环。
聊天记忆隔离	MemorySaver + 固定 thread_id=123	学习原型存在跨用户状态混用风险，生产必须按用户会话隔离。

常见误区

- **“有 Agent 就会自动操作所有系统”**：不对；只可使用 tools 参数传入的能力。
- **“Prompt 要求确认，所以写入一定安全”**：不对；当前没有服务端确认门禁。
- **“菜单为空就是模型坏了”**：不对；菜单 API、MySQL、聊天模型、Milvus 分属不同链路。
- **“SSE 是 WebSocket”**：不是；是单个 HTTP 响应的流式传输。
- **“ToolMessage 就是用户看到的回答”**：不是；它是工具结果，当前后端过滤后由模型再总结。
- **“保存到购物车就是下单了”**：不是；当前并无完整订单提交接口。

12. 本地运行与调试提纲

启动顺序

1. 确认项目根目录 `.env` 中模型 endpoint/key/model 与 MySQL 配置正确，勿将密钥贴到聊天或提交到仓库。
2. 根目录启动后端： `uv run uvicorn api.main:app --reload --host 127.0.0.1 --port 8010` 。
3. 另开终端，在 `ui` 目录执行 `npm run dev` ；访问 Vite 终端报告的 Local URL。
4. 浏览器 Network 面板确认 `POST /api/chat` 状态、响应 Content-Type 为 `text/event-stream` ，并检查是否持续收到 `data:` 行。

按层定位错误

现象	定位层	检查项
浏览器 ECONNREFUSED 到 8010	本地进程/代理	后端是否监听代理目标端口，Vite proxy target 是否一致。
`/health` 正常但聊天报错	模型配置/网络	OPENAI_*、地域 endpoint、证书、上游权限/额度与模型 ID；health 是存活探针，不证明模型可用。
菜单能看但助手不推荐	Agent 工具路由	检查模型是否选择工具、ToolMessage 结果、工具返回是否为空。
向量失败而菜单回退正常	Milvus 依赖	检查本地模型、URI/token、collection/schema；MySQL 回退不是 Milvus 修复。
前端收到 error SSE	异步流/异常映射	查看后端 traceback；浏览器只收到安全提示，不包含原始上游密钥/响应。

自动化验证

本仓库有 `tests/test_assistant.py` 。从根目录运行：

```
uv run python -m unittest discover -s tests -v
```

测试覆盖 SSL 证书校验、配置错误、菜单 MySQL 回退、Milvus 异常和 SSE 错误脱敏；单元测试不等于真实模型/MCP 的集成测试。任何真实聊天都会访问上游模型并消耗账号额度。

13. 进阶练习（从观察到改造）

1. **观察无工具回复：**问营业时间，在日志确认模型回答但没有 ToolMessage。
2. **观察 MySQL Tool：**问“列出特色菜和价格”，识别模型产生的 tool call、函数 SQL 与工具结果。
3. **观察回退：**让本地 BGE/Milvus 配置不可用，在仅测试环境确认 `user_flavor_search` 返回 MySQL 数据。
4. **阅读 Schema：**说明 Pydantic 字段如何映射到模型工具参数；提出人数和时间校验规则。
5. **设计确认门禁：**为预订引入服务端确认 state/token，使未确认情况下函数拒绝写入。
6. **修复线程隔离：**让每个浏览器会话有独立 thread_id，并验证多个用户历史不能互相读取。
7. **安全的取消预约：**设计状态字段和身份/预约所有权校验，先保留记录，不做任意 ID 硬删除。
8. **MCP 沙盒：**用测试 MCP server 验证工具发现、失败回退、权限和超时；不要一开始接生产写工具。
9. **前端断流：**模拟代理断开，检查 Reader 释放/取消、用户消息及助手状态是否正确。

> **副作用提醒：**工具可读写数据库，Agent 自动选工具不代表该调用经过用户界面单独审批。新工具上线前要按“参数校验 → 身份授权 → 确认状态 → 幂等 → 事务 → 审计 → 测试”逐项设计。

14. 源码索引与术语表

文件	建议阅读重点
`agent/langchain_assistant.py`	模型客户端、Engine、LangChain tools、回退、Agent 构造和 SSE 生成器。
`agent/prompts/system_prompt.txt`	模型的业务角色、可回答范围、预订指引、工具路由与现实边界。
`api/main.py`	FastAPI app/lifespan、Pydantic model、/chat、/menu/list、/reservation/list、Redis FAQ 路由。
`ui/src/api/index.js`	前端 Fetch 流读取、SSE 行解析及菜单/预订 API wrapper。
`ui/src/app.vue`	聊天状态、消息显示、按钮/回车事件、菜单和预订列表加载。
`ui/vite.config.js`	开发端口与 /api 代理。
`agent/milvus_data_sync.py`	学习向量数据同步流程；执行前审查删除 collection 的副作用。
`tests/test_assistant.py`	连接、错误处理、MySQL 回退与流式生成器测试。

术语	本项目语境中的意思
LLM / Chat Model	接收消息并生成回复或工具调用请求的上游聊天模型。
Tool	由 Python 实际实现、带有供模型调用 Schema 的函数。
Tool Call	模型请求运行某工具及参数；不是数据库操作本身，运行时才执行函数。
ToolMessage	本地或 MCP 工具执行后的结果消息，返回 Agent 继续推理。
ReAct	模型交替进行思考决策与行动（调用工具）的 Agent 模式；最终给出答复。
State / Checkpoint	图运行中的消息等数据，以及按 thread_id 保存的线程状态。
MCP Client/Server	Client 发现并调用 Server 提供的协议化工具。
SSE	单个 HTTP 响应中逐帧发送的事件流；此处帧由 data: JSON 组成。
Fallback	可选向量检索失败时，将查询路径切换为 MySQL 菜单读取，不是模型供应商切换。

> **最后的阅读建议：**遇到一个 Agent 行为时，依次追问：“Prompt 允许什么？模型选择了哪个 Tool？Schema 限制什么？Python 函数实际做什么？外部系统写入了什么？结果如何传回 UI？”这套问题能帮助你把 LLM 决策和程序执行分清。

本学习指南描述生成时仓库中可见的实现。端点、模型默认值、服务配置或功能若后来发生变化，应重新对照源码核验。